

Development of an objective value optimization-based traversal algorithm for a graph generation framework

Péter András Tömöri

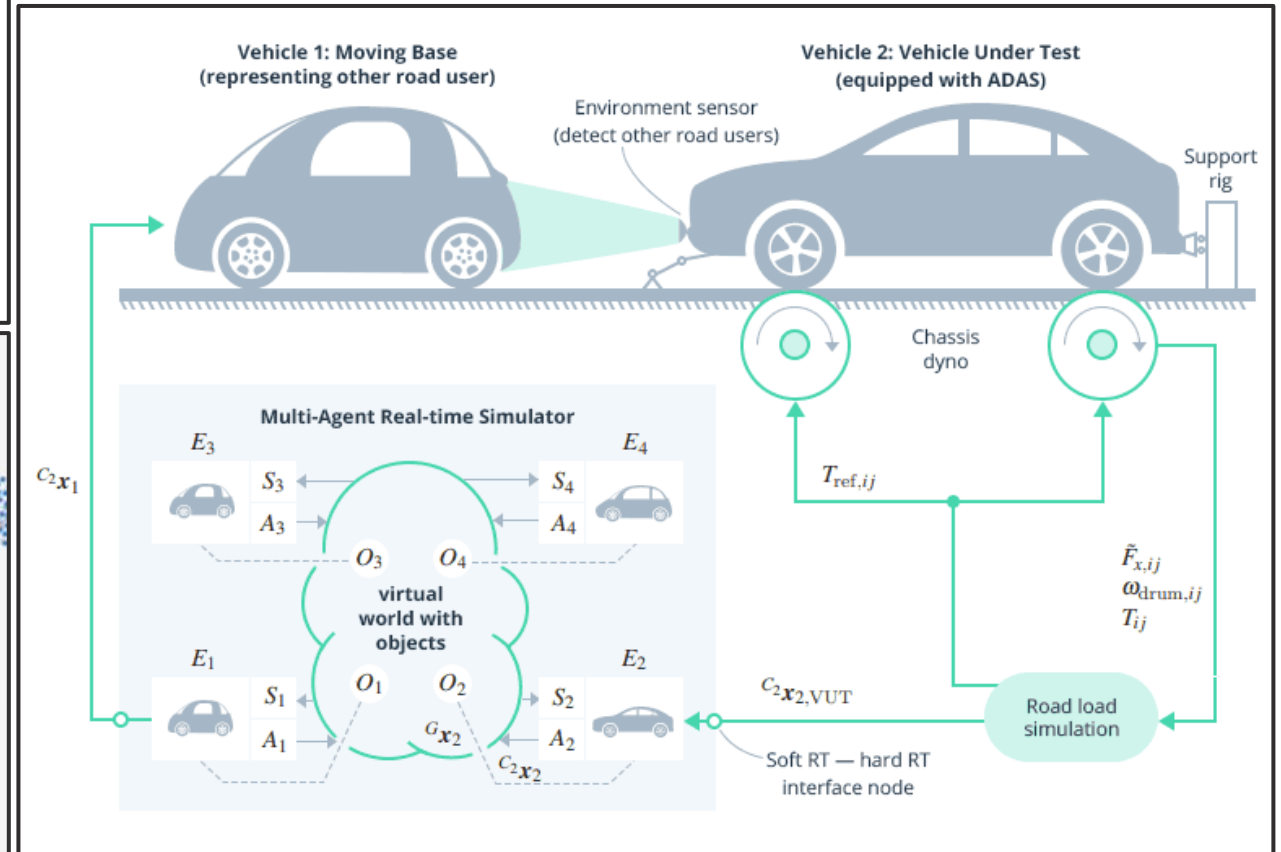
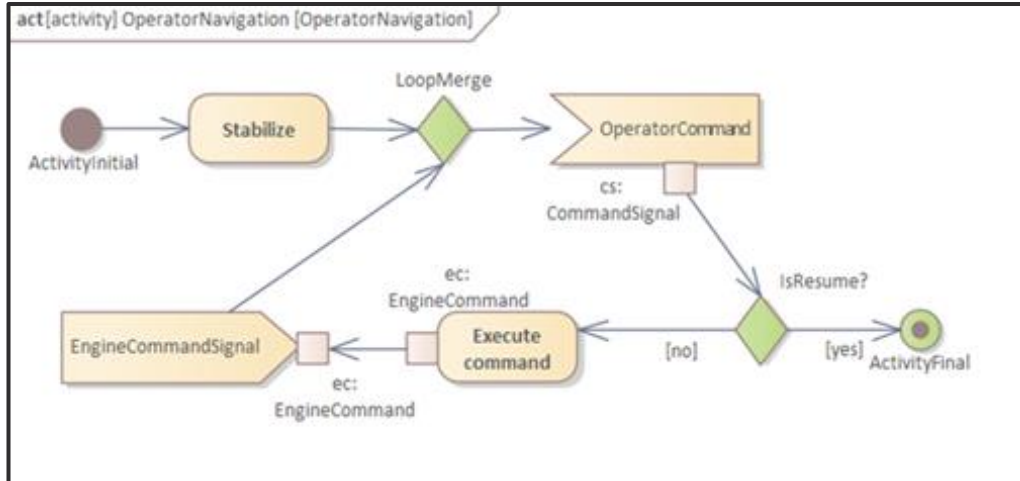
Supervisors: Attila Ficsor, Oszkár Semeráth, Kristóf Marussy



**Critical Systems
Research Group**

Motivation

consistent | realistic | diverse | scalable

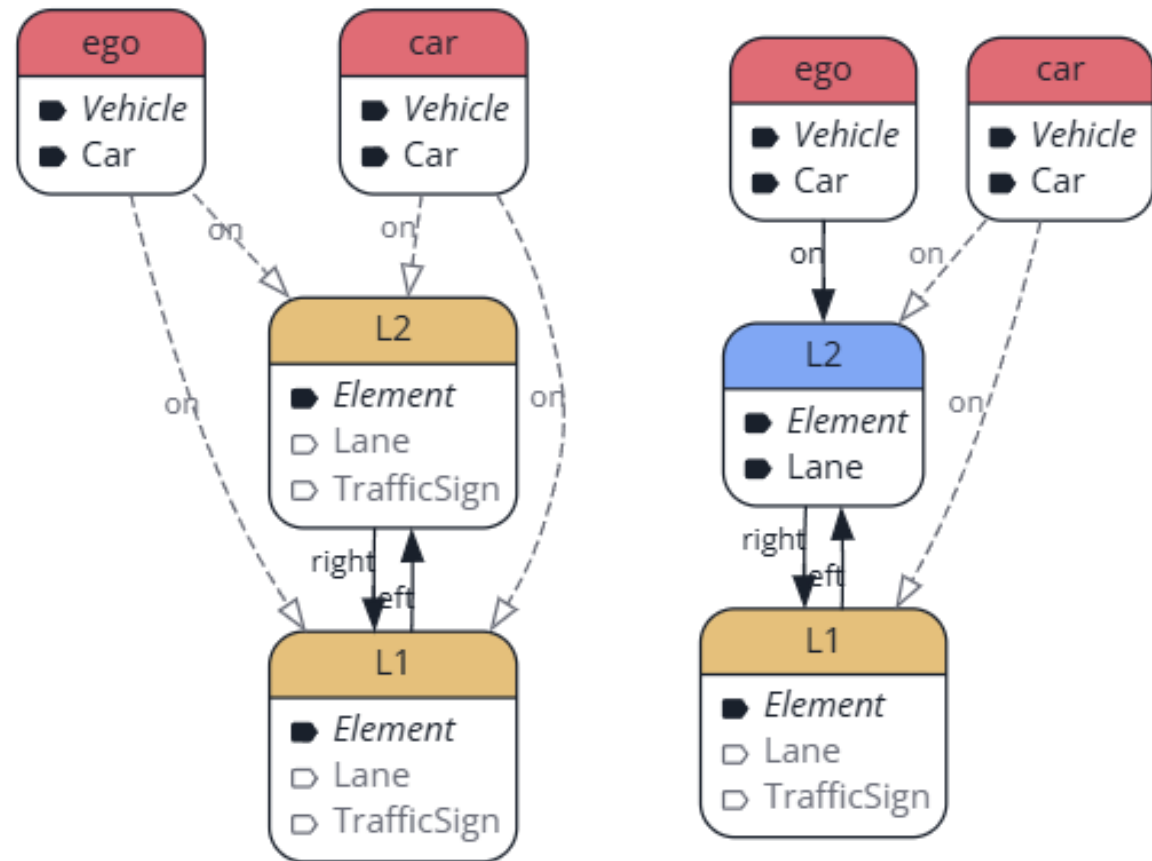


<https://neo4j.com/graph-visualization-neo4j/> <https://intellias.com/three-ways-of-testing-ad-as-in-autonomous-cars-beyond-a-test-drive/>

Preliminary in Refinery

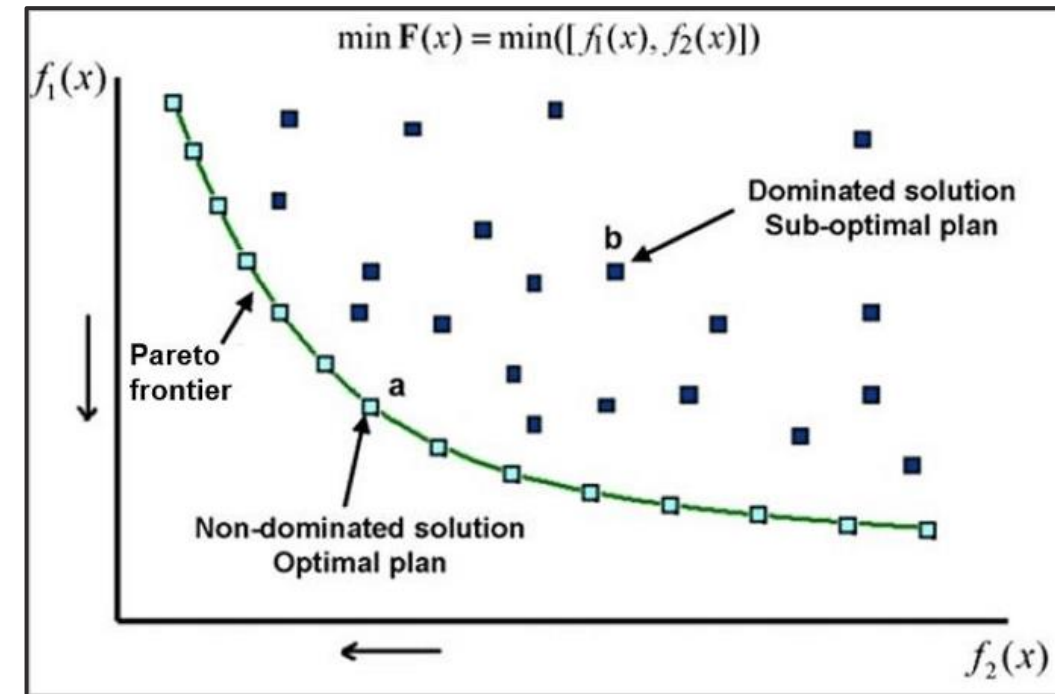


- Design Space Exploration
 - **Exhaustive** | **Heuristic** | Stochastic | Evolutionary
- Partial Modeling
 - Refinement
 - 4-valued logic
- Decision Rules
- Unit Propagation Rules



Challenges

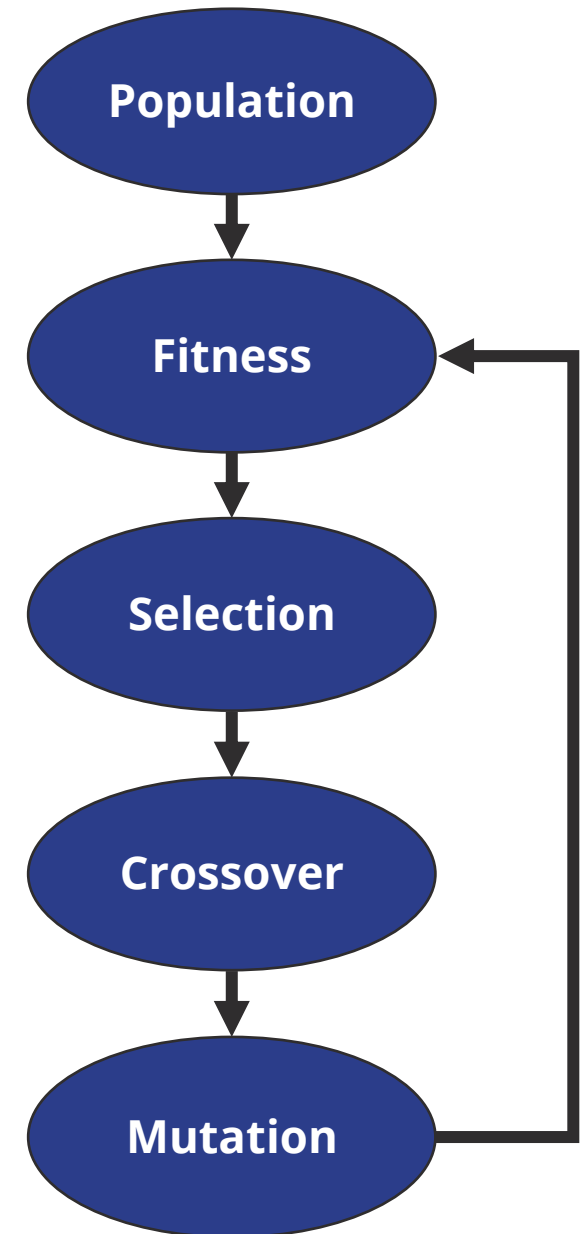
- Large combinatorial design space
- Optimization with specific objectives
 - time / quantity / diversity
- Exploration of complete Pareto-front
 - Multi-objective optimization



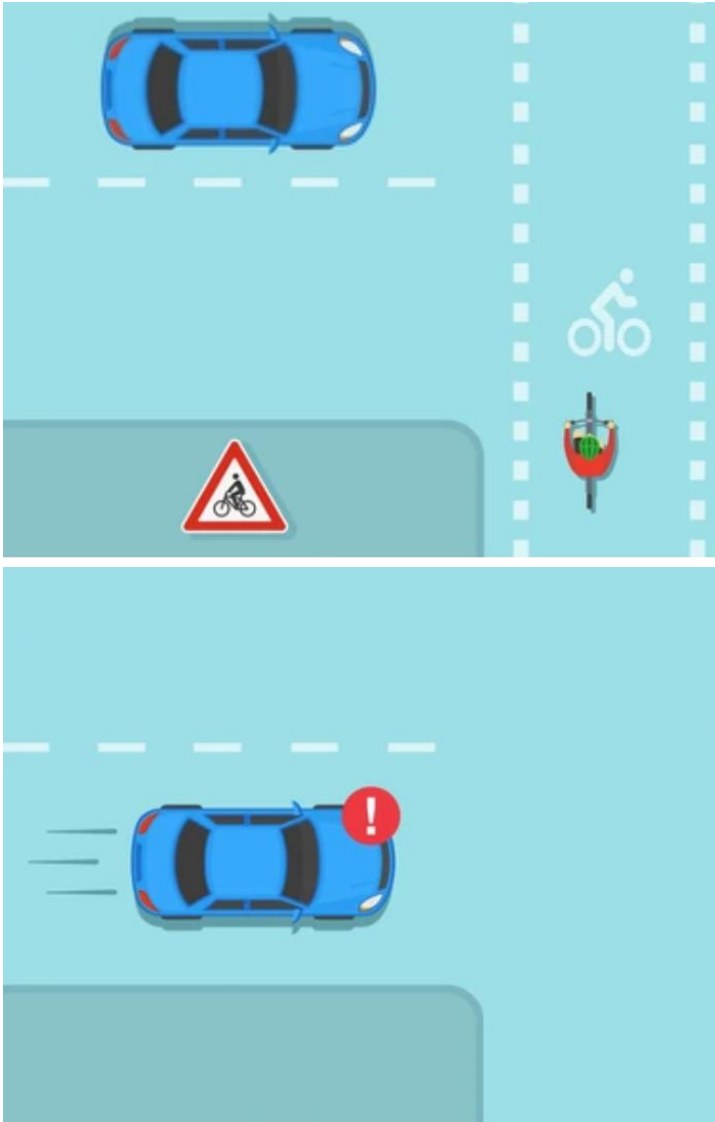
https://www.researchgate.net/figure/Example-of-a-Pareto-frontier-for-a-multicriteria-objective-optimization-problem-with-two_fig6_309609991

Genetic algorithms

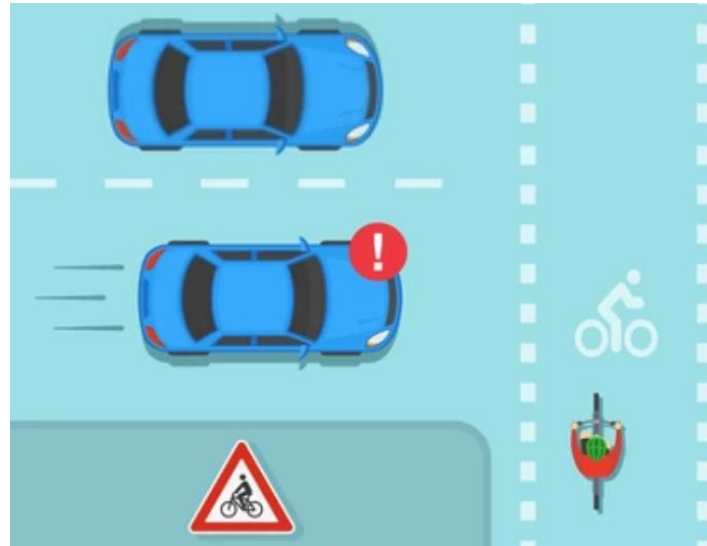
- Based on biological evolution and natural selection
- Suitable for optimization problems
- AGE2-MOEA, MOEA-D, NSGA-II, SPEA2 etc.
- Dominance, elitism, crowding distance, penalty function, repair operator



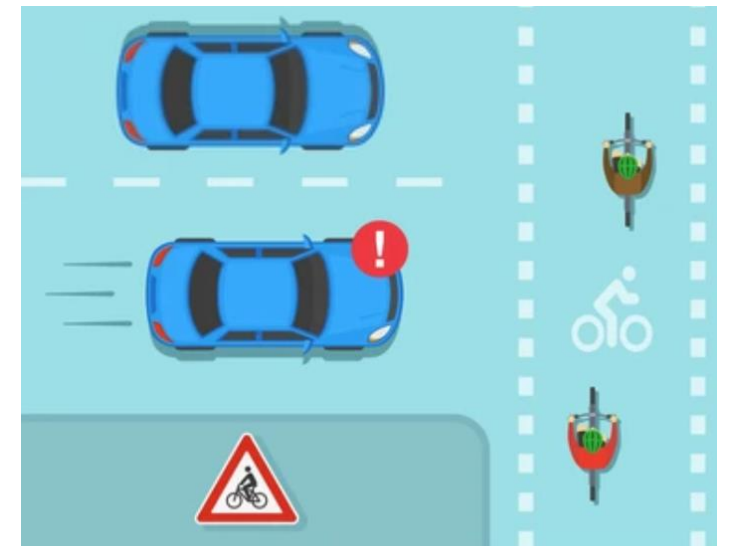
Genetic algorithms



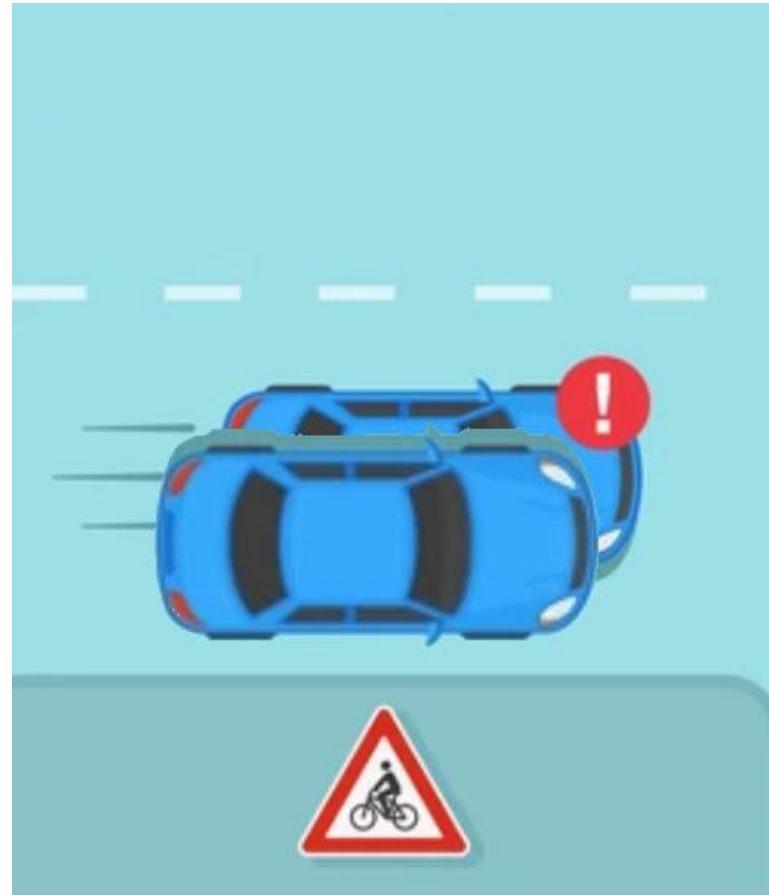
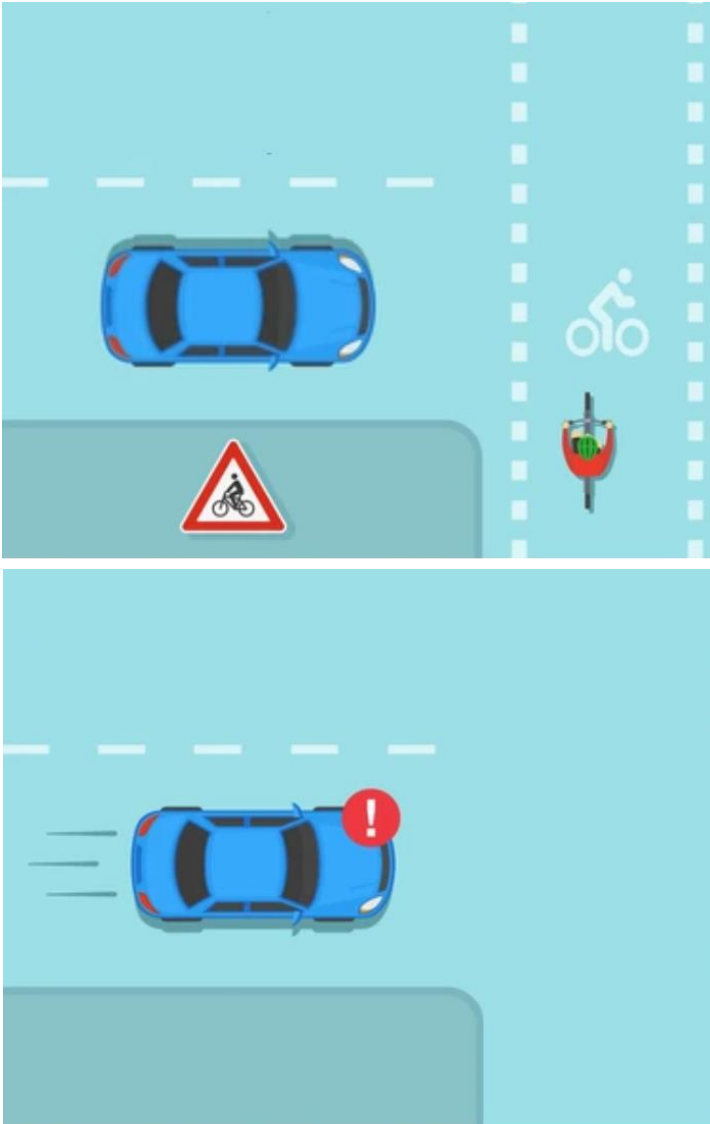
Crossover



Mutation

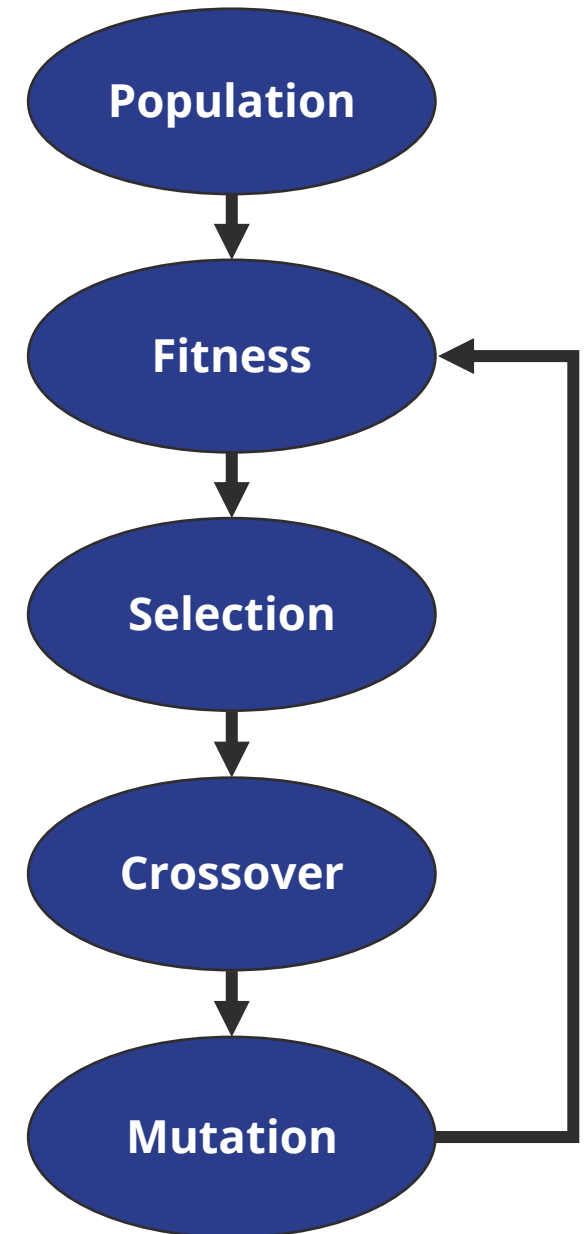


Challenge: Crossover of Graphs



Genetic algorithms

- Based on biological evolution and natural selection
- Suitable for optimization problems
- AGE2-MOEA, MOEA-D, NSGA-II, SPEA2 etc.
- Dominance, elitism, crowding distance, penalty function, repair operator



MOEA integration to Refinery

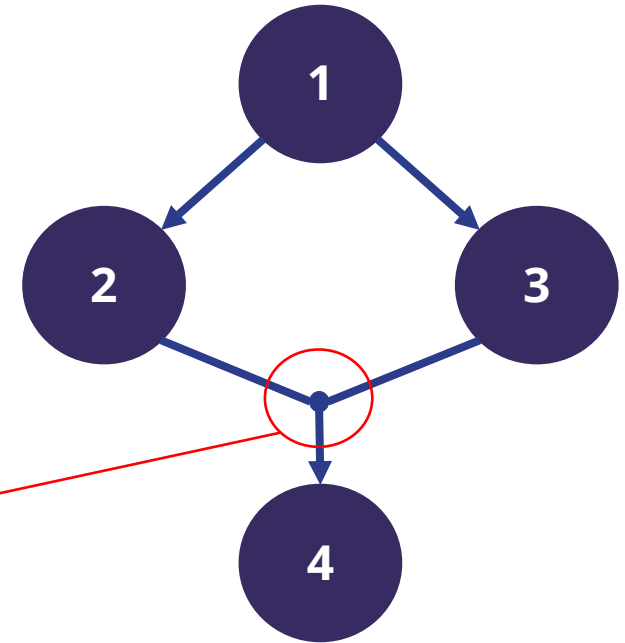


- Rich selection of algorithms
- Well documented API, tested and efficient implementations
- Multi-objective and Pareto-optimality support
- RefineryProblem with VersionVariable and RuleBasedMutation
 - Test: Class Responsibility Assignment Index
- Related Work: MOMoT
 - Marrying Search-Based Optimization and Model Transformations
 - Use of MOEA, but more focused on transformations rather than graph modeling

Summary

1. Understanding main concepts of graph generation and DSE
2. Getting familiar with Refinery framework
3. Advantages of evolutionary algorithms
4. Integration of MOEA Framework

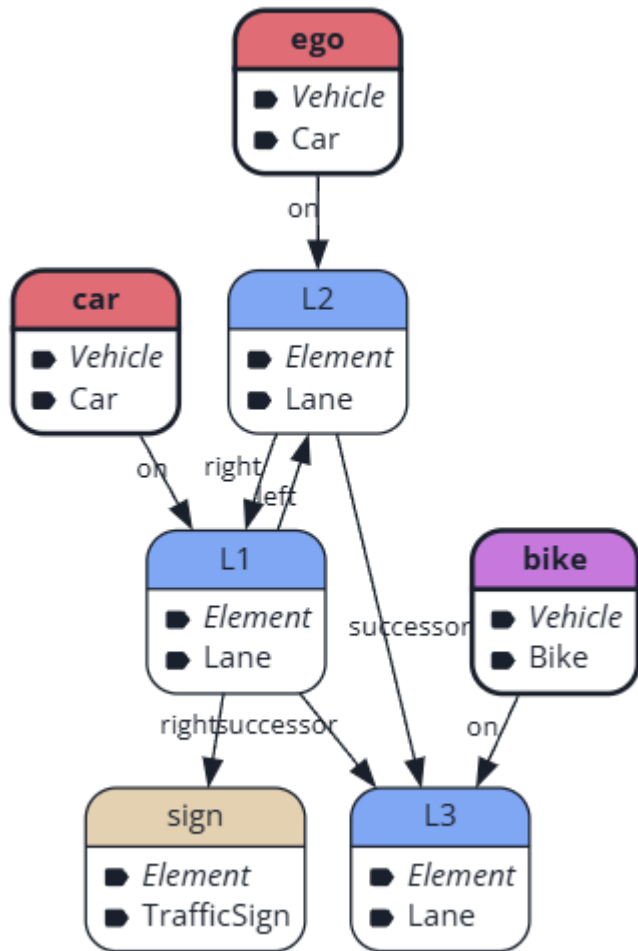
Future work: **Crossover** function of graphs
Evaluation of different algorithms



References

- Oszkár Semeráth, András Szabolcs Nagy, and Dániel Varró. 2018. A graph solver for the automated generation of consistent domain-specific models. In Proceedings of the 40th International Conference on Software Engineering (ICSE '18). Association for Computing Machinery, New York, NY, USA, 969–980. <https://doi.org/10.1145/3180155.3180186>
- <https://refinery.tools/>
- <https://navisp.esa.int/project/details/294/show>
- Máté Földiák, Kristóf Marussy, Dániel Varró, and István Majzik. 2022. System Architecture Synthesis for Performability by Logic Solvers. In ACM/IEEE 25th International Conference on Model Driven Engineering Languages and Systems (MODELS '22), October 23–28, 2022, Montreal, QC, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3550355.3552448>
- <https://moeaframework.org/>
- Hadka, D. (2025). MOEA Framework A Free and Open Source Java Framework for Multiobjective Optimization (Version 5.0) [Computer software]. Retrieved from <https://github.com/MOEAFramework/MOEAFramework>.
- <https://ceur-ws.org/Vol-1758/paper1.pdf>

Traffic situation example



shutterstock.com · 2175267373

```
private static final RelationalQuery carOnLane = Query.of("CarOnLane", (builder, c1, l1) -> builder.clause(
    carView.call(c1), laneView.call(l1), c1.notEquivalent(l1), onTrueView.call(c1, l1)
));
```

```
private static final RelationalQuery inOneLine = Query.of("InOneLine", (builder, o1, o2) -> builder
    .clause(carOnLane.call(o1, o2))
    .clause(carOnLane.call(o2, o1))
    .clause(l -> List.of(carOnLane.call(o1, l), carOnLane.call(o2, l))));
```

```
private static final RelationalQuery leftTo = Query.of("LeftTo", (builder, o1, o2) -> builder.clause(
    leftTrueView.callTransitive(o1, o2)
));
```

```
private static final Rule assignLeftBetweenLines = Rule.of("AssignLeftBetweenLines", (builder, a, b) ->
    builder
        .clause((c, d) -> List.of(
            not(inOneLine.call(a, b)),
            inOneLine.call(a, c), inOneLine.call(b, d),
            leftTo.call(c, d), not(leftTo.call(a, b)))
        )
        .action(
            put(left, 1.0, a, b)
        ));
```

```
private static final Symbol<Boolean> car = Symbol.of("Car", 1);

private static final Symbol<Double> on = Symbol.of("On", 2,
    Double.class, null);

private static final AnySymbolView carView = new
    KeyOnlyView<>(car);
private static final AnySymbolView laneView = new
    KeyOnlyView<>(lane);

private static final AnySymbolView onView = new
    FunctionView<>(on);
private static final AnySymbolView leftView = new
    FunctionView<>(left);
private static final AnySymbolView onTrueView = new
    FilteredView<>(on, value -> value == 1.0);
private static final AnySymbolView leftTrueView = new
    FilteredView<>(left, value -> value == 1.0);
```

Algorithm	Type	List of algorithms						
		Real	Binary	Permutation	Subset	Grammar	Program	Constraints
AGE-MOEA-II	Genetic Algorithm	Yes	Yes	Yes	Yes	Yes	Yes	Yes
AMOSa	Simulated Annealing	Yes	Yes	Yes	Yes	Yes	Yes	Yes
CMA-ES	Evolutionary Strategy	Yes	No	No	No	No	No	Yes
DBEA	Decomposition	Yes	Yes	Yes	Yes	Yes	Yes	Yes
DE	Single Objective	Yes	No	No	No	No	No	Yes
eMOEA	ε -Dominance	Yes	Yes	Yes	Yes	Yes	Yes	Yes
eNSGAI	ε -Dominance	Yes	Yes	Yes	Yes	Yes	Yes	Yes
ES	Single Objective	Yes	No	No	No	No	No	Yes
GA	Single Objective	Yes	Yes	Yes	Yes	Yes	Yes	Yes
GDE3	Differential Evolution	Yes	No	No	No	No	No	Yes
IBEA	Indicator-Based	Yes	Yes	Yes	Yes	Yes	Yes	No
MOEAD	Decomposition	Yes	Yes	Yes	Yes	Yes	Yes	Yes
MSOPS	Differential Evolution	Yes	No	No	No	No	No	Yes
NSGAI	Genetic Algorithm	Yes	Yes	Yes	Yes	Yes	Yes	Yes
NSGAIII	Reference Points	Yes	Yes	Yes	Yes	Yes	Yes	Yes
OMOPSO	Particle Swarm	Yes	No	No	No	No	No	Yes
PAES	Evolutionary Strategy	Yes	Yes	Yes	Yes	Yes	Yes	Yes
PESA2	Genetic Algorithm	Yes	Yes	Yes	Yes	Yes	Yes	Yes
RSO	Single Objective	Yes	Yes	Yes	Yes	Yes	Yes	Yes
RVEA	Reference Vectors	Yes	Yes	Yes	Yes	Yes	Yes	Yes
SA	Simulated Annealing	Yes	Yes	Yes	Yes	Yes	Yes	Yes
SMPSO	Particle Swarm	Yes	No	No	No	No	No	Yes
SMSEMOA	Indicator-Based	Yes	Yes	Yes	Yes	Yes	Yes	Yes
SPEA2	Genetic Algorithm	Yes	Yes	Yes	Yes	Yes	Yes	Yes
UNSGAIII	Reference Points	Yes	Yes	Yes	Yes	Yes	Yes	Yes
VEGA	Genetic Algorithm	Yes	Yes	Yes	Yes	Yes	Yes	No

```
RefineryProblem problem = new RefineryProblem(store, initialVersion, 5);
NSGAI algorithm = new NSGAI(problem);
```

```
algorithm.setVariation(problem.getMutation());
algorithm.setInitialPopulationSize(10);
```

```
algorithm.run(10000);
NondominatedPopulation result = algorithm.getResult();
```

CRA-Index = CohesionRatio - CouplingRatio

$$CohesionRatio = \sum_{c_i \in Classes} \frac{MAI(c_i, c_i)}{|M(c_i)| \times |A(c_i)|} + \frac{MMI(c_i, c_i)}{|M(c_i)| \times |M(c_i) - 1|}$$

$$CouplingRatio = \sum_{\substack{c_i, c_j \in Classes \\ c_i \neq c_j}} \frac{MAI(c_i, c_j)}{|M(c_i)| \times |A(c_j)|} + \frac{MMI(c_i, c_j)}{|M(c_i)| \times |M(c_j) - 1|}$$

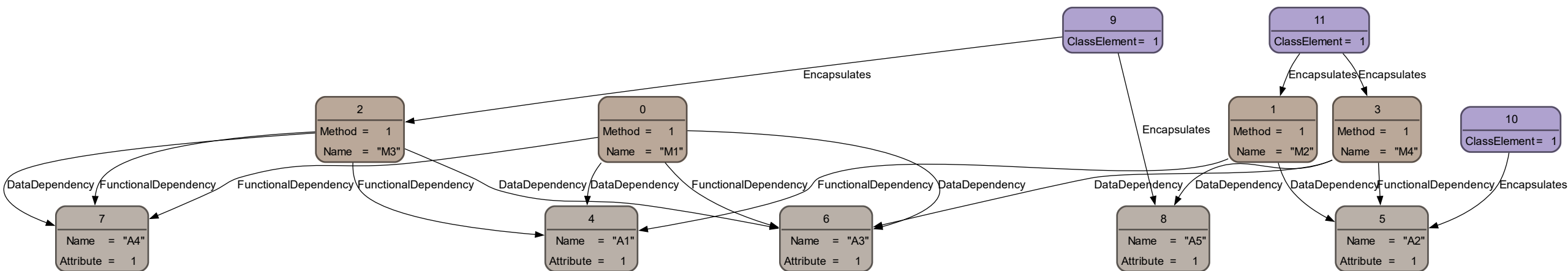
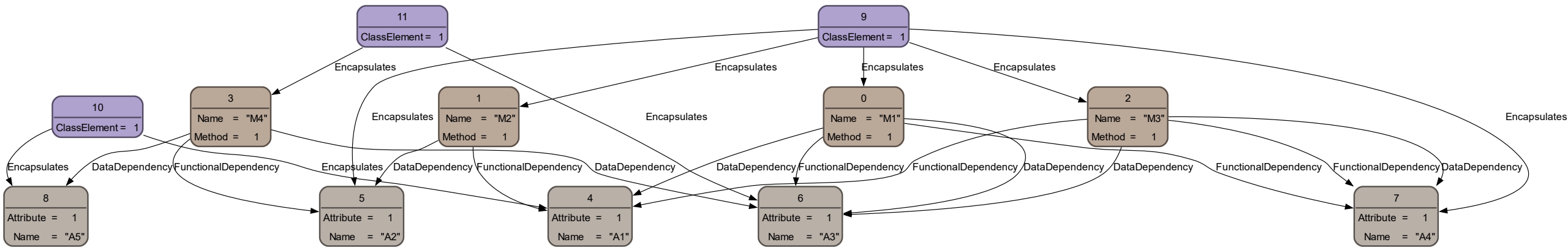
$$MMI(c_i, c_j) = \sum_{\substack{m_i \in M(c_i) \\ m_j \in M(c_j)}} DMM(m_i, m_j)$$

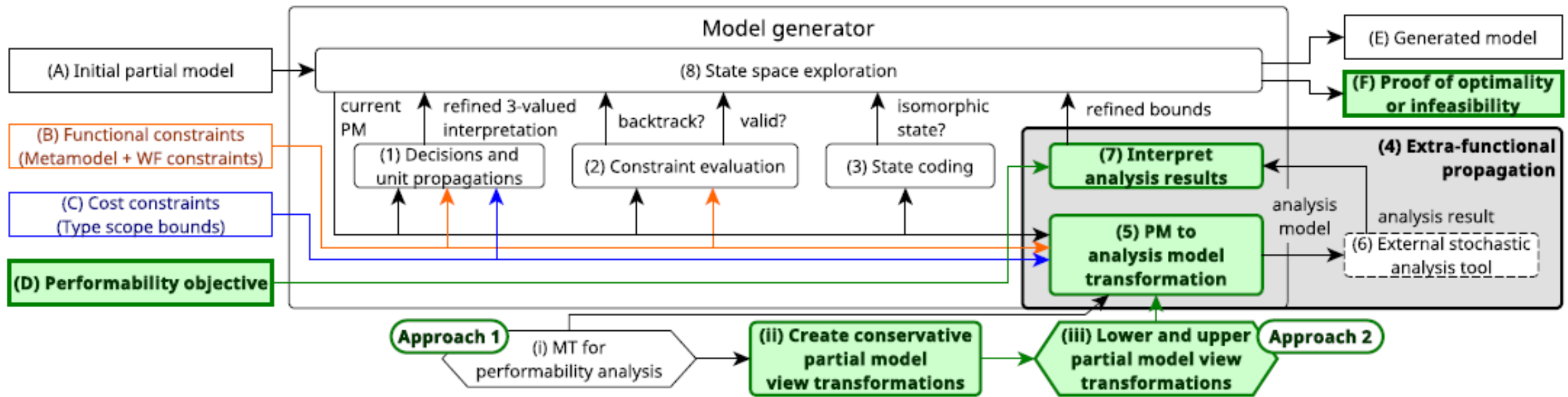
$$MAI(c_i, c_j) = \sum_{\substack{m_i \in M(c_i) \\ a_j \in A(c_j)}} DMA(m_i, a_j)$$

$$DMA(m_i, a_j) = \begin{cases} 1 & \text{if there is a dependency between method } m_i \text{ and attribute } a_j \\ 0 & \text{otherwise} \end{cases}$$

$$DMM(m_i, m_j) = \begin{cases} 1 & \text{if there is a dependency between method } m_i \text{ and } m_j \\ 0 & \text{otherwise} \end{cases}$$

```
var classCohesion = Query.of("classCohesion", Double.class,
    (builder, c, output) -> builder.clause(Integer.class, mc -> List.of(
        classElementView.call(c),
        mc.assign(methodCount.leftJoin(0, c)),
        output.assign(RealTerms.add(
            RealTerms.div(
                RealTerms.asReal(mai.leftJoin(0, c, c)),
                RealTerms.asReal(IntTerms.mul(
                    mc,
                    attributeCount.leftJoin(0, c)
                ))
            ),
            RealTerms.div(
                RealTerms.asReal(mmi.leftJoin(0, c, c)),
                RealTerms.asReal(IntTerms.mul(
                    mc,
                    IntTerms.sub(mc, IntTerms.constant(1))
                ))
            )
        )))
    ));
```





// Objective 1: number of on and left that are not 1.0 or 0.0

```
private static final RelationalQuery unsureRelations =
Query.of("unsureRelations", (builder, a, b) -> builder
    .clause(onUnsureView.call(a, b))
    .clause(leftUnsureView.call(a, b)));

private static final FunctionalQuery<Integer> numOfUnsureRelations =
Query.of(Integer.class,
    (builder, output) -> builder
    .clause(
        output.assign(unsureRelations.count(Variable.of(), Variable.of()))
    ));
```

```
private static final Rule setOnToOne =
Rule.of("SetOnToOne", (builder, a, b, v) -> builder
    .clause(
        onUnsureView.call(a, b)
    )
    .action(
        remove(on, a, b),
        put(on, 1.0, a, b)
        // put(loss, 1.0 - v)
    )
);
```